

Assignment

①

Q1) Find Solution of Linear System

a. Write a code taking as input a matrix A of size $m \times n$ and a vector b of size $m \times 1$, where m and n are arbitrarily large numbers and $m < n$, constructing the augmented matrix and performing * REF and RREF

without using any built-in functions. In case you encounter any division by 0, you can choose a different A and/or b .

Deliverables: The code snippet showing the procedure for REF and RREF.

A import numpy as np

m = int(input("Enter number of rows m (m < n): "))

n = int(input("Enter number of columns n: "))

if m >= n:

print("m should be less than n.")

→ $A = np.random.randint(-5, 6, (m, n))$ // integers between -5 and 5

Matrix A (4 x 5) with random (-5 to 5):

$$A = \begin{bmatrix} -1 & 0 & 3 & 5 & -3 \\ 4 & -3 & 3 & -4 & -3 \\ -4 & 1 & -4 & 5 & 1 \\ -2 & 4 & 0 & 0 & -1 \end{bmatrix}$$

$$\begin{cases} m = 4 \\ n = 5 \end{cases}$$

→ Random Seed vector x -seed ($m \times 1$) with integers bet³ -3 to 3

→ x -Seed = $np.random.randint(-3, 4, (m, 1))$

Print(x -Seed.flatten())

x -Seed = $\begin{bmatrix} 0 & 0 & 2 & 2 & -1 \end{bmatrix}$

→ $b = A @ x$ -seed

Print(b .flatten())

$$\begin{bmatrix} 1 & 9 & 1 & 1 & 1 \end{bmatrix}$$

(2)
Augmented matrix $[A|b]$

```
Aug = np.hstack((A.astype(float), b.astype(float)))
```

```
for row in Aug:
```

```
    print([round(x, 2) for x in row])
```

$$A|b = \begin{bmatrix} -1.0 & 0.0 & 3.0 & 5.0 & -3.0 & 19.0 \\ 4.0 & -3.0 & 3.0 & -4.0 & -3.0 & 11.0 \\ -4.0 & 1.0 & -4.0 & 5.0 & 1.0 & 1.0 \\ -2.0 & 4.0 & 0.0 & 0.0 & -1.0 & 1.0 \end{bmatrix}$$

REF function (Row Echelon form)

```
def REF(M):
```

```
    rows, cols = M.shape
```

```
    r = 0
```

```
    for c in range(cols-1):
```

```
        # pivot = r  
        # Search for non-zero pivot in current column
```

```
        print(f"\n Step: Looking for pivot in column {c}")
```

```
        while pivot < rows and abs(M[pivot][c]) < 1e-10:
```

```
            pivot += 1
```

```
        # Skip column if no non-zero pivot found
```

```
        if pivot == rows:
```

```
            print(f" No pivot found in column {c}, Skip row.")
```

```
            continue
```

```
        # Swap rows if needed
```

```
        if pivot != r:
```

```
            print(f" Swapping row {r} with row {pivot}")
```

```
            M[[r, pivot]] = M[[pivot, r]]
```

```
            pivot_value = M[r][c]
```

```
            print(f" pivot found at row {r}, column {c} with  
                  value: {pivot_value:.2f}")
```


(2)
e(float))

Eliminate entries below pivot
for i in range($r+1$, rows):
if $\text{abs}(M[i][c]) > 1e-10$:
factor = $M[i][c] / \text{pivot-value}$
print(f "row $\{i\} = \text{row } \{i\} - (\text{factor} \cdot \text{row } \{r\})$ "
 $\{r\}$ ")
 $M[i] = M[i] - \text{factor} \times M[r]$

$r \neq 1$

Stop if we've processed all rows
if $r \geq \text{rows}$:
break

Clean up very small numbers
for i in range(rows):
for j in range(cols):
if $\text{abs}(M[i][j]) < 1e-10$:
 $M[i][j] = 0.0$

return M
ref_matrix X = REF(AugCopy())
REF Computation Steps

Step: Looking for pivot in column 0

→ pivot found at row 0, column 0 with value: -1.00

→ row 1 = Row 1 - (-4.00) × row 0

→ row 2 = row 2 - (4.0) × row 0

→ row 3 = row 3 - 2 × row 0

Step: Looking for pivot in column 1

→ pivot found at row 1, column 1 with value -3.00

→ Row 2 = Row 2 - (-0.33) × row 1

→ row 3 = row 3 - (-1.33) × row 1

Step: Looking for pivot in column 2

→ pivot found at row 2, column 2 with value: -11.00

→ row 3 = row 3 - (-1.27) × row 2

Step: Looking for pivot in column 3

→ pivot found at row 3, column 3 with value: -0.97

RREF function (Reduced row Echelon form)

(4)

```
def RREF(M):
```

```
    # First convert to REF
```

```
    M = REF(M.copy())
```

```
    rows, cols = M.shape
```

```
    # work from bottom row upward
```

```
    for i in range(rows - 1, -1, -1):
```

```
        # Find pivot column (First non-zero in current row)
```

```
        pivot_col = -1
```

```
        for j in range(cols - 1):
```

```
            if abs(M[i][j]) > 1e-10:
```

```
                pivot_col = j
```

```
                break
```

```
    # Skip if no pivot in this row
```

```
    if pivot_col == -1:
```

```
        print(f"\n row {i}: No pivot found, skipping...")
```

```
        continue
```

```
    print(f"\n row {i}: processing pivot and column {pivot_col}")
```

```
    # Normalize pivot row (make pivot = 1)
```

```
    pivot_value = M[i][pivot_col]
```

```
    if abs(pivot_value) > 1e-10:
```

```
        print(f"Normalizing row {i}: dividing by {pivot_value:.2f}")
```

```
        M[i] = M[i] / pivot_value
```

```
    # Eliminate entries above pivot
```

```
    for k in range(i):
```

```
        if abs(M[k][pivot_col]) > 1e-10:
```

```
            factor = M[k][pivot_col]
```


Print ("row {k} = row {k} - ({factor:.2f})
X Row {i}")

$$M[k] = M[k] - \text{factor} \times M[i]$$

* Clean up very small numbers
for i in range(rows):

for j in range(cols):

if abs(M[i][j]) < 1e-10:

$$M[i][j] = 0.0$$

Return M

ref_matrix = RREF(Aug.copy())

Compute RREF Steps

Row 3: processing pivot at column 3

→ Normalizing row 3: dividing by -0.97

→ Row 0 = Row 0 - (5.00) X Row 3

→ Row 1 = Row 1 - (16.00) X Row 3

→ Row 2 = Row 2 - (-9.67) X Row 3

Row 2: processing pivot at column 2

→ Normalizing row 2: dividing by -11.00

→ Row 0 = Row 0 - (3.00) X Row 2

→ Row 1 = Row 1 - (15.00) X Row 2

Row 1: processing pivot at column 1

→ Normalizing row 1: dividing by -3.00

Row 0: processing pivot at column 0

→ Normalizing row 0: dividing by -1.00

Result RREF

for i, row in enumerate(ref_matrix):

print(f"row {i}: {[round(x, 4) for x in row]}")

$$\begin{bmatrix} -1 & 0 & 3 & 5 & -3 & 19 \\ 0 & -3 & 15 & 16 & -15 & 77 \\ 0 & 0 & -11 & -9 & 8 & -49.3333 \\ 0 & 0 & 0 & -0.9697 & -4.8182 & 2.8788 \end{bmatrix}$$

Result REF

(6)

for i, row in enumerate (ref-matrix):

print(f"row {i}: {[round(x,4) for x in row]}")

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 12.5625 & -12.5625 \\ 0 & 1 & 0 & 0 & 6.0312 & -6.0312 \\ 0 & 0 & 1 & 0 & -5.0937 & 7.0937 \\ 0 & 0 & 0 & 1 & 4.9687 & -2.9687 \end{bmatrix}$$

(1b) Identify the pivot and non-pivot columns and find the particular solution and solution to $Ax = 0$.

Deliverables: The Code Snippet Showing the pivot and non-pivot columns, particular solution and the solution to $Ax = 0$

A import numpy as np

A = np.array([
[-1, 0, 3, 5, -3], [4, -3, 3, -4, -3],
[-4, 1, -4, 5, 1], [-2, 4, 0, 0, -1]])

Matrix A from Q1(a)

* Vector b from Q1(a)

b = np.array([[19], [77], [-49.3333], [2.8788]])

RREF of Augmented Matrix $[A|b]$ from Q1 (a)

$\text{rref_augmented} = \text{np.array}([$
[1.0, 0.0, 0.0, 0.0, 12.5625, -12.5625],
[0.0, 1.0, 0.0, 0.0, 6.0312, -6.0312],
[0.0, 0.0, 1.0, 0.0, -5.0937, 7.0937],
[0.0, 0.0, 0.0, 1.0, 4.9687, -2.9687]])

Extract RREF of A and b

$\text{rref_A} = \text{rref_augmented}[:, :5]$

$\text{rref_b} = \text{rref_augmented}[:, 5:6]$

~~rref_A~~
$$\begin{bmatrix} 1 & 0 & 0 & 0 & 12.5625 \\ 0 & 1 & 0 & 0 & 6.0312 \\ 0 & 0 & 1 & 0 & -5.0937 \\ 0 & 0 & 0 & 1 & 4.9687 \end{bmatrix}$$

$$\begin{bmatrix} -12.5625 & -6.0312 & 7.0937 & -2.9687 \end{bmatrix}$$

Step 1 identity pivot and Non pivot columns

$\text{rows, cols} = \text{rref_A}.shape$

$\text{pivot_cols} = []$

$\text{non_pivot_cols} = []$

for i in range(rows):

for j in range(cols):

if $\text{abs}(\text{rref_A}[i][j] - 1.0) < 1e-4$:

$\text{pivot_cols.append}(j)$

break

for j in range(cols):

if j not in pivot_cols:

$\text{non_pivot_cols.append}(j)$


```
print(f"pivot columns (basic variables): {pivot_cols}")
print(f"Non-pivot columns (free variables): {non_pivot_cols}")
```

Step-2 Find particular solution ($Ax=b$)

```
x_particular = np.zeros(cols)
```

```
# Set free variable to 0
```

```
for col in non_pivot_cols:
```

```
    x_particular[col] = 0.0
```

```
# Solve for basic variable
```

```
for idx, col in enumerate(pivot_cols):
```

```
    x_particular[col] = xref - b[idx][0]
```

```
for i in range(cols):
```

```
    if i in pivot_cols:
```

```
        print(f"x{i} = {x_particular[i]:.4f} (basic)")
```

```
    else:
```

```
        print(f"x{i} = {x_particular[i]:.4f} (free)")
```

```
print(f"x_p = {[round(x, 4) for x in x_particular]}")
```

Step-3 Find null Space Basis (solution to $Ax=0$)

```
null_vector = []
```

```
for free_col in non_pivot_cols:
```

```
    x_null = np.zeros(cols)
```

```
    x_null[free_col] = 1.0
```


for row in range(rows-1, -1, -1):

 pivot-in-row = -1

 for j in range(cols):

 if abs(ref_A[row][j]-1.0) < 1e-4:

 pivot-in-row = j

 break

 if pivot-in-row != -1 and pivot-in-row != free_col:

 x-null[pivot-in-row] = -ref_A[row][free_col]

 null-vectors.append(x-null)

for i in range(cols):

 if i in pivot-cols:

 print(f"x{i} = {null-vectors[0][i]:.4f} (basic)")

 else:

 print(f"x{i} = {null-vectors[0][i]:.4f} (free=1)")

print(f"1st v = {[round(x,4) for x in null-vectors[0]]}")

Result:

Step-1 pivot and not pivot columns.

pivot column: [0,1,2,3]

Not pivot " : [4]

Step-2 (Ax=b)

$x_0 = -12.5625$ (basic)

$x_1 = -6.0312$ (")

$x_2 = 7.0937$ (")

$x_3 = -2.9687$ (")

$x_4 = 0.0000$ (Free)

Step-3

$x_0 = -12.5625$

$x_1 = -6.0312$

$x_2 = 7.0937$

$x_3 = -2.9687$

$x_4 = 1.0000$

$V = [-12.5625, -6.0312, 7.0937, -2.9687, 1]$

$V = [-12.5625, -6.0312, 7.0937, -2.9687, 0]$

④c) Consider, a random 6×9 matrix A and (10)
a suitable b and show the REF, RREF,
Pivot Columns, non-pivot Columns, the
Particular Solution, the Solution to $Ax = b$,
the general solution and verify the general
Solution.

Deliverables: The random matrix A , vector b
and other quantities mentioned in the question.

① import numpy as np

Step 1 (generate random 6×9 matrix and (consistent
 b))

np.random.seed(42) // everytime generate random
number for every run we can
ignore also

$A = \text{np.random.randn}(-5, 6, (6, 9))$

$x\text{-Seed} = \text{np.random.randint}(-3, 4, (9, 1))$

$b = A @ x\text{-Seed}$

print(A)

print(b.flatten())

Step 2 Compute REF and RREF

def REF(M):

rows, cols = M.shape

$M = M.copy().astype(float)$

$r = 0$

for c in range(cols-1):

if $r \geq \text{rows}$:

break

pivot = r

while pivot < rows and $\text{abs}(M[\text{pivot}][c])$
< $1e-10$:


```

    pivot += 1
    if pivot == rows:
        continue
    if pivot != r:
        M[[r, pivot]] = M[[pivot, r]]
    pivot_val = M[r][c]
    for i in range(r+1, rows):
        if abs(M[i][c]) > 1e-10:
            M[i] = M[i] - (M[i][c] / pivot_val) * M[r]
    r += 1
    for i in range(rows):
        for j in range(cols):
            if abs(M[i][j]) < 1e-10:
                M[i][j] = 0.0

```

return M

```
def RREF(M):
```

```

    rows, cols = M.shape
    M = M.copy().astype(float)
    r = 0

```

forward elimination to REF

```
for c in range(cols-1):
```

```

    if r >= rows:
        break

```

```
    pivot = r
```

```

    while pivot < rows and abs(M[pivot][c]) < 1e-10:

```

```
        pivot += 1
```

```
    if pivot == rows:
```

```
        continue
```

```
    if pivot != r:
```


$$M[r, \text{pivot}] = M[\text{pivot}, r] \quad (12)$$

$$\text{pivot_val} = M[r][c]$$

for i in range($r+1$, rows):

if $\text{abs}(M[i][c]) > 1e-10$:

$$M[i] = M[i] - (M[i][c] / \text{pivot_val}) * M[r]$$

$$r += 1$$

Backward elimination to RREF

for i in range(rows-1, -1, -1):

$$\text{pivot_col} = -1$$

for j in range(cols-1):

if $\text{abs}(M[i][j]) > 1e-10$:

$$\text{pivot_col} = j$$

break

if $\text{pivot_col} == -1$:

continue

$$M[i] = M[i] / M[i][\text{pivot_col}]$$

for k in range(i):

if $\text{abs}(M[k][\text{pivot_col}]) > 1e-10$:

$$M[k] = M[k] - M[k][\text{pivot_col}] * M[i]$$

for i in range(rows):

for j in range(cols):

if $\text{abs}(M[i][j]) < 1e-10$:

$$M[i][j] = 0.0$$

return M

create augmented matrix

$$\text{Aug} = \text{np.hstack}((A.\text{astype}(\text{float}), b.\text{astype}(\text{float})))$$

compute RREF

$$\text{ref_matrix} = \text{RREF}(\text{Aug})$$

(12)

```
for i in range(6):
    print (f"R{i+1}: {[round(ref_matrix[i][j], 4) for j in range(10)]}")
```

Compute RREF

(13)

```
rref_matrix = RREF(Aug)
```

```
for i in range(6):
    print (f"R{i+1}: {[round(rref_matrix[i][j], 4) for j in range(10)]}")
```

Extract rref-A and rref-b

```
rref_A = rref_matrix[:, :9]
```

```
rref_B = rref_matrix[:, 9:10]
```

Step-3 (Pivot and non pivot column)

```
pivot_cols = []
```

```
for i in range(6):
    for j in range(9):
        if abs(rref_A[i][j] - 1.0) < 1e-4:
            pivot_cols.append(j)
```

```
        break
```

```
non_pivot_cols = [j for j in range(9) if j not in pivot_cols]
```

```
print(pivot_cols)
```

```
print(non_pivot_cols)
```

Step-4 (Particular Solution ($Ax=b$))

```
x_p = np.zeros(9)
```

```
for idx, col in enumerate(pivot_cols):
```

```
    if idx < len(rref_b):
```

```
        x_p[col] = rref_b[idx][0]
```

```
print (f"x_p = {[round(x, 4) for x in x_p]}")
```


Step-5 (null Space Basis (solution to $Ax=0$))

(14)

```
null_vectors = []
```

```
for free_col in non_pivot_cols:
```

```
    v = np.zeros(9)
```

```
    v[free_col] = 1.0
```

```
    for row, pivot in enumerate(pivot_cols):
```

```
        v[pivot] = -xref_A[row][free_col]
```

```
    null_vectors.append(v)
```

```
print("\n -
```

```
for p, v in enumerate(null_vectors):
```

```
    print(f"v $\{i+1\}$  = {[round(x, 4) for x in v]}")
```

```
print(f"\ndimension of null space = {len(null_vectors)}")
```

Step-6 (General Solution)

```
if len(null_vectors) == 1:
```

```
    print("x = x-p + c.v $_1$ , where  $c \in \mathbb{R}$ ")
```

```
elif len(null_vectors) > 1:
```

```
    terms = ""
```

```
    for i in range(len(null_vectors)):
```

```
        print(f"x = x-p + {terms}, where  $c_1, \dots, c_2 \in \mathbb{R}$ ")
```

```
for i in range(9):
```

```
    if i in non_pivot_cols:
```

```
        idx = non_pivot_cols.index(i)
```

```
        print(f"{i} = c $\{idx+1\}$ ")
```

```
else:
```

```
    eqn = f"x $\{i\}$  = {x-p[i]:.4f}"
```

```
    for j, v in enumerate(null_vectors):
```

```
        if abs(v[i]) > 1e-6:
```

```
            sign = "+" if v[i] > 0 else "-"
```

```
            eqn += f"{sign}{v[i]:.4f}c $\{j+1\}$ "
```

```
    print(eqn)
```


Step-7 verify general solution (15)

```
c-test = np.random.randn(len(null-vectors))
```

```
x-test = x-p-copy()
```

```
for i, c in enumerate(c-test):
```

```
    x-test += c * null-vectors[i]
```

```
Ax-test = A @ x-test
```

```
print(f"for random constant c = {[round(c,4) for c in c-test]}:")
```

```
print(f"A * (x-p + Σ(c_i v_i)) = {[round(x,4) for x in Ax-test]}")
```

```
print(f"b = {[round(x,4) for x in b.flatten()]}")
```

```
if np.allclose(Ax-test, b.flatten(), atol=1e-4):
```

```
    print("Verified general solution is correct")
```

```
else:
```

```
    print("Verification failed")
```

Output

-- Matrix A (6x9) --

```
[[1 -2 5 2 -1 1 4 -3 1]
```

```
[-5 5 2 -1 -2 2 2 -3 0]
```

```
[-1 -4 2 0 -4 -1 -5 4 0]
```

```
[3 -5 5 5 4 -3 1 -2 3]
```

```
[-3 -1 -3 1 -1 3 1 -4 -2]
```

```
[3 -4 4 3 4 -1 -4 -2 1]]
```

-- Vector b (6x1) --

```
[12 8 -11 26 -14 20]
```


-- REF of $[A|b]$ --

(16)

$$R_1: (1.0), (-2.0), (5.0), (2.0), (-1.0), (1.0), (4.0), (-3.0), (1.0), (12.0)$$

$$R_2: (0.0), (15.0), (-23.0), (-11.0), (8.0), (-3.0), (-18.0), (12.0), (-5.0), (-7.0)$$

$$R_3: (0.0), (0.0), (-2.2), (-2.4), (-3.8), (-1.2), (-8.2), (-8), (-1.0), (-19.8)$$

$$R_4: (0.0), (0.0), (0.0), (8.9697), (21.4242), (-1.1818), (21.7576), (-16.1212), (4.1818), (69.6667)$$

$$R_5: (0.0), (0.0), (0.0), (0.0), (-5.9454), (3.973), (-1.2973), (-3.1802), (-2.1351), (-17.4324)]$$

$$R_6: (0.0), (0.0), (0.0), (0.0), (0.0), (3.8665), (-2.0227), (-4.483), (-2.2301), (-2.6335)]$$

-- REF of $[A|b]$ --

$$R_1: (1.0), (0.0), (0.0), (0.0), (0.0), (0.0), (-2.1573), (0.0553), (-0.1974), (-0.1105)$$

$$R_2: (0.0), (1.0), (0.0), (0.0), (0.0), (0.0), (2.7136), (-0.5286), (0.38), (2.47)$$

$$R_3: (0.0), (0.0), (1.0), (0.0), (0.0), (0.0), (1.3262), (-0.086), (0.3204), (3.776)$$

$$R_4: (0.0), (0.0), (0.0), (1.0), (0.0), (0.0), (2.6725), (-1.3807), (0.453), (1.765)$$

$$R_5: (0.0), (0.0), (0.0), (0.0), (1.0), (0.0), (-0.1314), (-0.2384), (-0.0263), (2.4769)$$

$$R_6: (0.0), (0.0), (0.0), (0.0), (0.0), (1.0), (0.5231), (-1.1594), (-0.5768), (-0.6811)$$

-- Pivot columns - $[0, 1, 2, 3, 4, 5]$

-- Non-pivot column - $[6, 7, 8]$

-- Particular Solution $(Ax=b)$

$$x-p = [(-0.1105), (2.0572), (3.1719), (1.7615), (2.4787), (-0.6811), (0.0), (0.0), (0.0)]$$

-- Null Space Basis (solution to $Ax=0$)

$$v_1 = [(2.1533), (-2.7136), (1.3262), (-2.6725), (0.7314), (0.5231), (0.0), (0.0), (0.0)]$$

$$v_2 = [(-0.0553), (0.5286), (0.086), (1.3807), (0.2384), (1.1594), (0.0), (0.0), (0.0)]$$

$$v_3 = [(0.1974), (-0.38), (-0.3204), (-0.453), (0.0263), (0.5768), (0.0), (0.0), (0.0)]$$

general solution

where $C_1, C_2, \dots \in \mathbb{R}$

17) Consider a symmetric and positive definite matrix A of size $n \times n$. write a code to construct an elementary matrix for every elementary row operation that is performed on A and using the theory explained in the class write the decomposition of A as LU , where L is a lower triangular matrix and U is an upper triangular matrix. Deliverables: The code snippet showing the generation of elementary matrices for a given elementary row operation and getting L and U , and verification of $A = LU$.

18) import numpy as np

Step-1 Generate matrix

$n = \text{int}(\text{input}(\text{"Enter size of matrix n: "})) // 3$
 $\text{np.random.seed}(12)$ // we can ignore if want random integers

$B = \text{np.random.randint}(-5, 6, (n, n))$

$A = B.T @ B + n * \text{np.eye}(n)$

$A = (A + A.T) / 2$

for row in A:

print(" ", [round(x, 2) for x in row])

Output

(24.0), (-16.0), (11.0)

(-16.0), (17.0), (-14.0)

(11.0), (-14.0), (30.0)

Step-2 function to create elementary matrix

def elementary_matrix(n, row, col, multiplier):

""" $E = I$ with $-multiplier$ at (row, col) """

$E = \text{np.eye}(n)$

$E[row, col] = -multiplier$

return E

Step 3 function to multiply matrices

def matrix_multiply (X,Y):

m,n = X.shape

n,p = Y.shape

result = np.zeros (m,p)

for i in range(m):

for j in range(p):

for k in range(n):

result [i,j] += X[i,k] * Y[k,j]

return result

Step 4 function to invert lower triangular matrix

def invert_lower_triangular (L):

n = L.shape [0]

L_inv = np.eye (n)

for i in range(n):

for j in range(i):

Sum_K = 0

for k in range(j, i):

Sum_K += L[i,k] * L_inv[k,j]

L_inv[i,j] = -Sum_K / L[i,i]

return L_inv

Step 5 LU decomposition with elementary matrices

U = A.copy()

elementary_matrices = []

elimination_steps = []

Step 3 elimination process:

for col in range(n-1):

print (f"\n processing column {col}:")

for row in range (col+1, n):

if abs (U[row,col]) > 1e-10:

multiplier = U[row,col] / U[col,col]

print (f"R{row} - ({multiplier} * R{col})")


```

E = elementary_matrix(n, row, col, multiplier)
elementary_matrices.append(E)
elimination_steps.append((row, col, multiplier))
U = matrix_multiply(E, U)

```

(19)

Output Step-3

processing Column 0:

$$R_1 \rightarrow R_1 - (0.6667) \times R_0$$

$$R_2 \rightarrow R_2 - (0.4583) \times R_0$$

processing Column 1:

$$R_2 \rightarrow R_2 - (-1.0526) \times R_1$$

Step-6 Build L from product of elementary matrices
inverse

E-product = np.eye(n)

for E in elementary_matrices:

E-product = matrix_multiply(E, E-product)

L = invert_lower_triangular(E-product)

for row in L:

print(" ", [round(x, 4) for x in row])

for row in U:

print(" ", [round(x, 4) for x in row])

Step-7 verification $A = L \times U$ (m)

LU-product = matrix_multiply(L, U)

for row in LU-product:

print(" ", [round(x, 2) for x in row])

for row in A

print(" ", [round(x, 2) for x in row])

check if equal

diff = 0

for i in range(n):

for j in range(n):

diff = max(diff, abs(A[i, j] - LU-product[i, j]))

if diff < 1e-10:

print("verified: $A = L \times U$ ")

(20)

else: print(f"failed max diff = {diff:.2e}")

display elementary matrices

for i, (E, step) in enumerate(zip(elementary_matrices, elimination_steps)):

row, col, mult = step

print(f" $E \in \{i+1\}: R\{row\} \rightarrow R\{row\} - (\{mult\} \cdot R\{col\})$ ")

for r in range(n):

print(" ", [round(E[r][c], 4) for c in range(n)])

Step 4 output

Lower triangular

$[(1.0), (0.0), (0.0)]$

$[(-0.6667), (1.0), (0.0)]$

$[(0.4583), (-1.0526), (1.0)]$

Upper triangular

$[(24.0), (-16.0), (11.0)]$

$[(0.0), (6.3333), (-6.6667)]$

$[(0.0), (0.0), (17.9408)]$

Step 5 $A = L \times U$

$[(24.0), (-16.0), (11.0)]$

$[(-16.0), (17.0), (-11.0)]$

$[(11.0), (-14.0), (30.0)]$

original A:

$[(24.0), (-16.0), (11.0)]$

$[(-16.0), (17.0), (-11.0)]$

$[(11.0), (-14.0), (30.0)]$

Verified: $A = L \times U$

elementary matrices needed.

$E_1: R_1 \rightarrow R_1 - (-0.6667) \times R_0$

$[(1.0), (0.0), (0.0)]$

$[(0.6667), (1.0), (0.0)]$

$[(0.0), (0.0), (1.0)]$

$E_3: R_2 \rightarrow R_2 - (-1.0526) \times R_1$

$[(1.0), (0.0), (0.0)]$

$[(0.0), (1.0), (0.0)]$

$[(0.0), (1.0526), (1.0)]$

$E_2: R_2 \rightarrow R_2 - (0.4583) \times R_0$

$[(1.0), (0.0), (0.0)]$

$[(0.0), (1.0), (0.0)]$

$[(0.4583), (0.0), (1.0)]$

(2b) For the above problem, work out the Cholesky decomposition.

Deliverables: The code snippet showing the generation of L and verification of $A = LL^T$

(A) Matrix taken from 2a.

Step 1

$$A = \begin{bmatrix} 24 & -16 & 11 \\ -16 & 17 & -14 \\ 11 & -14 & 30 \end{bmatrix}$$

Step 2 Cholesky decomposition

def cholesky-decomposition(A):

n = A.shape[0]

L = np.zeros((n,n))

for i in range(n):

for j in range(i+1):

if i == j:

Diagonal element

Sum-Sq = 0

for k in range(i):

Sum-Sq += L[i][k]**2

L[i][i] = np.sqrt(A[i][i] - Sum-Sq)

print(f"L[{i}][{i}] = sqrt({A[i][i]:.2f} -

- {Sum-Sq:.4f}) = {L[i][i]:.4f}")

else:

off-diagonal element

Sum-Prod = 0

for k in range(i):

Sum-Prod += L[i][k] * L[j][k]

L[i][j] = (A[i][j] - Sum-Prod) /

L[j][j]

print(f"L[{i}][{j}] = ({A[i][j]:.2f} - {Sum-Prod:.4f}) / {L[j][j]:.4f} = {L[i][j]:.4f}")

Return L

L = ch

Step 3

for row

Step 4

def mat

m, n =

n, p =

resol

for

return

Step 5

L.T =

LLT.Prod

for

for

diff = 0

for i

for

if diff

pr

$L = \text{cholesky_decomposition}(A)$

(22)

#Step 3 Lower triangle from Cholesky

for row in L:

print(" ", [round(x, 4) for x in row])

#Step 4 multiply matrices

def matrix_multiply(x, y):

m, n = x.shape

n, p = y.shape

result = np.zeros((m, p))

for i in range(m):

for j in range(p):

for k in range(n):

result[i, j] += x[i, k] * y[k, j]

return result.

#Step 5

$L^{-T} = L^{-T}$

LLT Product = matrix_multiply(L, L^{-T})

for row in LLT Product:

print(" ", [round(x, 2) for x in row])

for row in A:

print(" ", [round(x, 2) for x in row])

diff = 0

for i in range(n):

for j in range(n):

diff = max(diff, abs(A[i, j] - LLT Product[i, j]))

if diff < 1e-10:

print("Verified A = L x L^T!")

Result

(23)

Step-2 Cholesky Algorithm process

$$L[0][0] = \text{Sqrt}(24.00 - 0.0000) = 4.8990$$

$$L[1][0] = (-16 - 0) / 4.8990 = -3.2660$$

$$L[1][1] = \text{Sqrt}(17.00 - 10.6667) = 2.5166$$

$$L[2][0] = 11.00 - 0 / 4.8990 = 2.2454$$

$$L[2][1] = -14.00 - -7.3333 / 2.5166 = -2.6491$$

$$L[2][2] = \text{Sqrt}(30.00 - 12.0000) = 4.2357$$

Step-3 Lower triangular from Cholesky

$$[4.899, 0, 0]$$

$$[-3.266, 2.5166, 0]$$

$$[2.2454, -2.6491, 4.2357]$$

Step-4 $L \times L^T$

$$\begin{bmatrix} 24 & -16 & 11 \\ -16 & 17 & -14 \\ 11 & -14 & 30 \end{bmatrix}$$

Now

$$L \times L^T = A$$

Verified.

(2c) Given n linearly independent vectors in m dimensions (the corresponding matrix with these as columns is A of size $m \times n$), with $m > n$, read about the QR decomposition of a matrix and generate Q and R .

Deliverables: The code snippet generating Q and R for the given A .

(A) import numpy as np

Step 1 Create Matrix with $m > n$

$$m, n = 6, 4$$

np.random.seed(42)

$A = \text{np.random.randint}(-5, 6, (m, n)).\text{astype}(\text{float})$

for row in A:
 print(" ", [round(x, 2) for x in row])

result :-

$[(1.0), (-2.0), (5.0), (2.0)]$
 $[(-1.0), (1.0), (4.0), (-3.0)]$
 $[(1.0), (5.0), (5.0), (2.0)]$
 $[(-1.0), (-2.0), (2.0), (2.0)]$
 $[(-3.0), (0.0), (-1.0), (-4.0)]$
 $[(2.0), (0.0), (-4.0), (-1.0)]$

Step 2: - Gram-Schmidt QR decomposition

def qr-decomposition(A):

$m, n = A.shape$

$Q = np.zeros((m, n))$

$R = np.zeros((n, n))$

for j in range(n):

$v = A[:, j].copy()$

Step 2: - function to compute norm

def vector-norm(v):

$norm_sq = 0$

for val in v:

$norm_sq += val**2$

return $np.sqrt(norm_sq)$

Step 3: function for dot product

def dot-product(v1, v2):

result = 0

for i in range(len(v1)):

result += $v1[i] * v2[i]$

return result

Step 4: multiply matrices

def matrix-multiply(x, y):

$m, n_x = x.shape$

$n_y, p = y.shape$

result = $np.zeros((m, p))$


```
for i in range(m):
```

```
    for j in range(p):
```

```
        for k in range(n-x):
```

```
            result[i,j] += x[p,k] * y[k,j]
```

~~Refactor~~

Return result

Step-5 QR decomposition

def qr_decomposition(A):

m,n = A.shape

Q = np.zeros((m,n))

R = np.zeros((n,n))

for j in range(n):

v = A[:,j].copy()

print(f"Initial v = {[round(x,4) for x in v]}")

for i in range(j):

R[i,j] = dot-product(Q[:,i], A[:,j])

print(f"R[{i}][{j}] = Q[:,{i}]^T * A[:,{j}] =
{R[i,j]:.4f}")

for k in range(m):

v[k] -= R[i,j] * Q[k,i]

print(f"After subtracting projection v =
{[round(x,4) for x in v]}")

R[j,j] = vector-norm(v)

print(f"R[{j}][{j}] = ||v|| = {R[j,j]:.4f}")

for k in range(m):

Q[k,j] = v[k] / R[j,j]

print(f"Q[:,{j}] = v / R[{j}][{j}] = {[round(x,4) for x in Q[:,j]]}")

Return Q,R

Q,R = qr_decomposition(A)

(25)

for row in Q:
 print(" ", [round(x,4) for x in row]) (26)

result: Orthogonal matrix 6x4

[(0.2425), (-0.38888), (0.6811), (-0.2241)]
 [(-0.2425), (0.2148), (0.3772), (-0.6966)]
 [(0.2425), (0.8287), (0.3203), (0.2262)]
 [(-0.2425), (-0.3069), (0.3081), (0.4414)]
 [(-0.7276), (0.1228), (-0.168), (-0.1969)]
 [(0.4851), (-0.0818), (-0.41), (-0.424)]

for row in R:
 print(" ", [round(x,4) for x in row])

result: Upper Triangular, 4x4

[(4.1231), (0.9701), (-0.2425), (3.638)]
 [(0.0), (5.7497), (2.6498), (-0.7878)]
 [(0.0), (0.0), (8.9398), (2.5694)]
 [(0.0), (0.0), (0.0), (4.1884)]

Step 6 Verification $A = QR$

QR-product = matrix_multiply(Q,R)

diff = 0

for i in range(m):

for j in range(n):

diff = max(diff, abs(A[i,j] - QR-product[i,j]))

if diff < 1e-10:

print("Verified: A = QR")

else:

print("Verification failed")

Step-7 Orthonormality check $Q^T Q = I$ (27)

$Q^T Q = \text{matrix-multiply}(Q.T, Q)$

for row in $Q^T Q$:

print(" ", [round(x, 4) for x in row])

#check if identity

is_identity = True

for i in range(n):

for j in range(n):

expected = 1 if $i == j$ else 0

if $\text{abs}(Q^T Q[i, j] - \text{expected}) > 1e-10$:

is_identity = False

if is_identity:

print("\n $Q^T Q = I$ (columns are orthonormal)")

else:

print("\n orthonormality check failed")

Result $Q^T Q$

[(0.0), (0.0), (-0.0), (0.0)]

[(0.0), (1.0), (-0.0), (0.0)]

[(-0.0), (-0.0), (1.0), (0.0)]

[(0.0), (0.0), (0.0), (1.0)]

(2d) Take a random 7x5 matrix having all its columns are linearly codependent and its decompose into Q and R, what is your observation on the diagonal element of R?

Deliverable: The rand matrix Q and R and your observation on the diagonal element of R.

(A) import numpy as np
np.random.seed(42)

(28)

A = np.random.randint(-10, 11, (7, 5)).astype(float)

for row in A:

print(" ", [round(x, 2) for x in row])

Step-2

Q, R = qr_decomposition(A) # function from Q2(1)

from row in Q:

print(" ", [round(x, 4) for x in row])

for row in R:

print(" ", [round(x, 4) for x in row])

Step-3 display diagonal elements of R

diagonal-R = [R[i][i] for i in range(5)]

for i, d in enumerate(diagonal-R):

print(f" R[{i}] [{i}] = {d : .6f}")

Step-4 observation

all-positive = all(d > 1e-10 for d in diagonal-R)

print(f" all diagonal entries positive: {all-positive}")

Results:-

Step-1 A =

$$\begin{bmatrix} 4 & 9 & 4 & 0 & -3 \\ 10 & -4 & 8 & 0 & 0 \\ 10 & -7 & -3 & -8 & 10 \\ -9 & 1 & -5 & -9 & 10 \\ -10 & 1 & 1 & 6 & -1 \\ 5 & 4 & 4 & 8 & 1 \\ 9 & -8 & -6 & 8 & -4 \end{bmatrix}$$

Step-2 decomposition Results

Q (7x5)

$$\begin{bmatrix} -0.1784 & 0.6275 & -0.1202 & -0.099 & -0.1668 \\ 0.4459 & 0.0271 & 0.7453 & -0.1743 & 0.0388 \\ 0.4459 & -0.2317 & -0.1787 & -0.4604 & 0.3829 \\ -0.4013 & -0.2487 & -0.2347 & -0.3891 & 0.4414 \\ -0.4459 & -0.2859 & 0.439 & 0.421 & 0.3595 \\ 0.2229 & 0.5311 & -0.0985 & 0.3385 & 0.7012 \\ 0.4013 & -0.3552 & -0.3752 & 0.5519 & -0.0914 \end{bmatrix}$$

R (5x5 Upper Triangular):

(29)

22.4277	-9.6765	1.5606	2.3632	0.0446
0	11.5924	8.6349	3.7839	-4.4485
0	0	9.487	2.3862	-2.8099
0	0	0	16.8346	-10.4884
0	0	0	0	9.4503

Observations:

→ All diagonal entries positive true. (70)

→ Reason for positivity :-

- * each diagonal entry $R[j][j]$ is the length (norm) of the residual vector after Substn projection onto previous Q columns
- * The residual vector = Column j - projection onto Span of previous columns
- * Since all columns of A are linearly independent, this residual can't be the Zero vector
- * Therefore its norm is strictly greater than Zero

* Implications :-

- * Positive diagonal means R is invertible
- * $\det(R)$ = product of diagonal entries > 0
- * QR decomposition is unique for matrices with full column rank.